

Multi-Camera 3D Pose Extraction of a Person Mopping

using three fisheye security cameras - towards a robot that shadows human cleaning motion

Progress report - October 2026

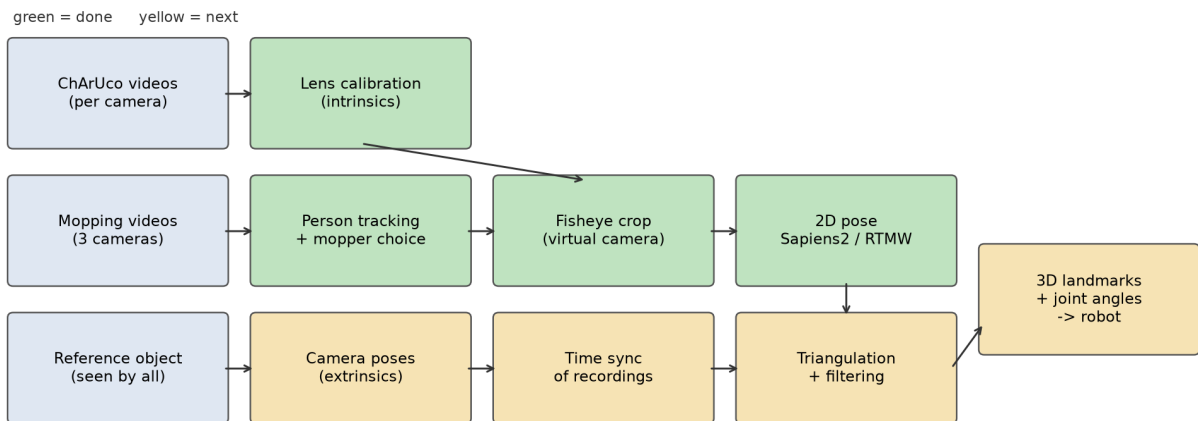


Figure 1 - Overall pipeline. Green: implemented and tested on the real recordings. Yellow: next steps (triangulation code already implemented and tested on synthetic data).

Summary

Three ceiling-mounted fisheye security cameras film a person mopping an office. The goal is a clean 3D trajectory of the body joints (and later joint angles) that a robot can imitate. So far: (1) each camera's fisheye lens is calibrated with a ChArUco board (reprojection RMS 0.76-0.95 px, covering 96-99 % of the area where people move); (2) a 2D pose pipeline finds the mopper among the other people, removes the fisheye distortion around them with a virtual pinhole camera and estimates 23 body + foot keypoints with two state-of-the-art models (Sapiens2 and RTMW) that agree within 1-4 px. Next: camera positions in a common world frame, time synchronisation and triangulation to 3D.

1. Goal and overall approach

The long-term aim is a robot that **tracks the movements of a person mopping**. The robot needs, for every moment in time, the 3D positions of the person's joints (shoulders, elbows, wrists, hips, knees, ankles, feet) in metres, and later the joint angles. This project builds the **measurement framework** that produces these landmarks from video; the robot control itself is a separate, later part.

Why several cameras? A single camera only sees a 2D projection: depth is ambiguous and body parts get hidden. If the same joint is seen from two or more calibrated cameras, its 3D position is the intersection of the viewing rays (*triangulation*). Three cameras also give redundancy when one view is occluded.

The pipeline (Figure 1) has these stages:

- **Intrinsic calibration** - per camera: focal length, image centre and fisheye distortion, so each pixel can be turned into a viewing ray. (*done*)
- **Person tracking** - find the mopper among the other people in the room. (*done*)
- **2D pose estimation** - body keypoints in every frame of every camera, robust to the fisheye distortion. (*done, full-length runs on a GPU computer*)
- **Extrinsic calibration** - position and orientation of each camera in one common world frame, from an object seen by all three cameras. (*next*)
- **Time synchronisation** - the cameras record independently; their videos must be aligned in time. (*next*)
- **Triangulation, filtering, joint angles** - 3D landmarks over time, smoothed, then a skeleton fit. (*triangulation implemented and tested; the rest next*)

Everything is implemented as one Python package (moppose) with a command-line tool, automated tests on synthetic data, and all results under version control (git).

2. Hardware and recordings

	value
Cameras	3 ceiling-mounted fisheye security cameras: cam1, cam3, cam4
Mopping videos	1280 x 1024 px, H.264 (.mkv), about 593 s each, recorded separately by each camera
Frame rate	file header says 15 fps, but the real rate is ~30 fps and variable (frame intervals 16-51 ms); dropped frames: cam1 3, cam3 4, cam4 23
Calibration board	ChArUco, A3 print, 3 x 2 squares (see section 3.2)
Board videos	one per camera, 158-210 s; cam3's was exported at 1920 x 1080 (see 3.4)
Computer	laptop with NVIDIA RTX 2070 (8 GB); heavy models run on a stronger GPU PC

Finding: the frame rate in the file cannot be trusted

Security cameras record with a variable frame rate and drop frames. Using frame_number / fps as the time would drift by seconds over 10 minutes and break the synchronisation of the three cameras. Therefore the framework reads the **real timestamp of every frame** from the video container and uses it everywhere.



Figure 2 - Frames of the cam1 mopping recording (time stamps in seconds). Strong fisheye distortion is visible at the image borders; several other people sit in the room.

3. Step 1 - Lens (intrinsic) calibration

3.1 What is calibrated and why

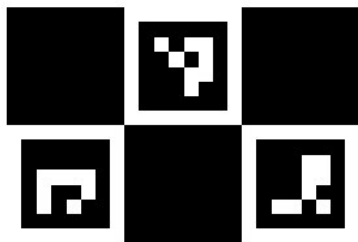
A camera maps a 3D direction to a pixel. For a normal (pinhole) lens a ray at angle θ from the optical axis lands at distance $r = f \cdot \tan \theta$ from the image centre; f is the focal length in pixels. A fisheye lens compresses wide angles to fit a large field of view, so the image is strongly curved. We use the **equidistant fisheye model** of OpenCV:

$$r(\theta) = f \cdot \theta \cdot (1 + k_1 \theta^2 + k_2 \theta^4 + k_3 \theta^6 + k_4 \theta^8)$$

The **intrinsic parameters** are the focal lengths f_x, f_y , the principal point (c_x, c_y) - where the optical axis hits the image - and the distortion coefficients $k_1..k_4$. Once known, every pixel can be converted into a 3D viewing ray, which is what triangulation needs. As a comparison the framework also fits the standard pinhole model with rational distortion (8 coefficients) and keeps the fisheye model unless the pinhole fit is clearly ($>10\%$) better.

Calibration works by filming a flat pattern with exactly known geometry (the board) in many positions. For each frame the detected board points in the image and their known positions on the board give equations; a non-linear least-squares optimisation finds the lens parameters (and the board pose in each frame) that minimise the **reprojection error**: the pixel distance between detected points and the points predicted by the model. Its root mean square (RMS) is the main quality number.

3.2 The ChArUco board



3 x 2 squares, square 115 mm, marker 86.2 mm, ArUco dictionary DICT_4X4_50 (marker ids 0-2), printed on A3. A ChArUco board combines a chessboard (precise corners) with ArUco markers (each corner can be identified even if the board is only partly visible).

The parameters were **verified against the PDF**: OpenCV's own board generator with these values reproduces the PDF pixel for pixel.

Problem 1 - a 3 x 2 board has only 2 chessboard corners

Calibration needs many well-spread points per frame. Solution: the 4 corners of each of the 3 markers are used as additional points (their 3D board positions are known too), giving **up to 14 points per frame**. Synthetic tests showed that with so few points per frame the quality comes from the number and variety of frames: ~40 frames gave ~0.45° ray error, ~100 frames ~0.15°.

Problem 2 - a half-pixel error inside OpenCV 4.11

On synthetic images with known ground truth, OpenCV's ChArUco detector returned the chessboard corners shifted by **+0.5 px** in x and y, while the marker corners were unbiased. Mixing both point types made the focal length 1.4 % wrong. The framework now measures this offset at start-up on a rendered board with known corners and subtracts it, so the correction adapts to whatever OpenCV version is installed. Afterwards the focal length was recovered within 1 %.

3.3 Calibration procedure (command `calib-intrinsics`)

- sample one frame every 0.25 s of the board video; detect the board
- reject frames with fewer than 8 points, motion blur (variance of the Laplacian below a threshold) or points nearly on a line (no geometric information)

- choose ~80 frames that cover the image evenly (greedy selection over an 8 x 8 grid with diminishing returns, and no two frames within 0.5 s)
- fit the fisheye and the pinhole model; iteratively drop frames whose own error is far above the median
- write a report: coverage map, per-frame errors, undistorted image, and a usability verdict

3.4 Different video formats

Some board videos were exported at 1920 x 1080 with black side bars, whereas the mopping videos are the camera's native 1280 x 1024. A calibration is only valid for one pixel grid, so the framework detects the active picture area, crops the bars and scales the board frames to 1280 x 1024 before detection. The mapping is verified automatically by matching the static room background (SIFT features) between the board video and the mopping video: the residual shift was **0.4-0.9 px**.

3.5 First attempt - and why it failed

In the first board recordings the board was held in the middle of the image only (it reached 39-46 % of the distance to the image corners). The RMS error looked excellent (0.48-0.72 px) - but only because the model had to fit the centre. Figure 3 shows the consequence for cam1: the fitted lens curve (orange, dashed) follows the true one up to ~30° and then **folds back**, which is physically impossible. Undistorting the image with it produced a black image. **A low RMS does not prove a good calibration; coverage matters.**

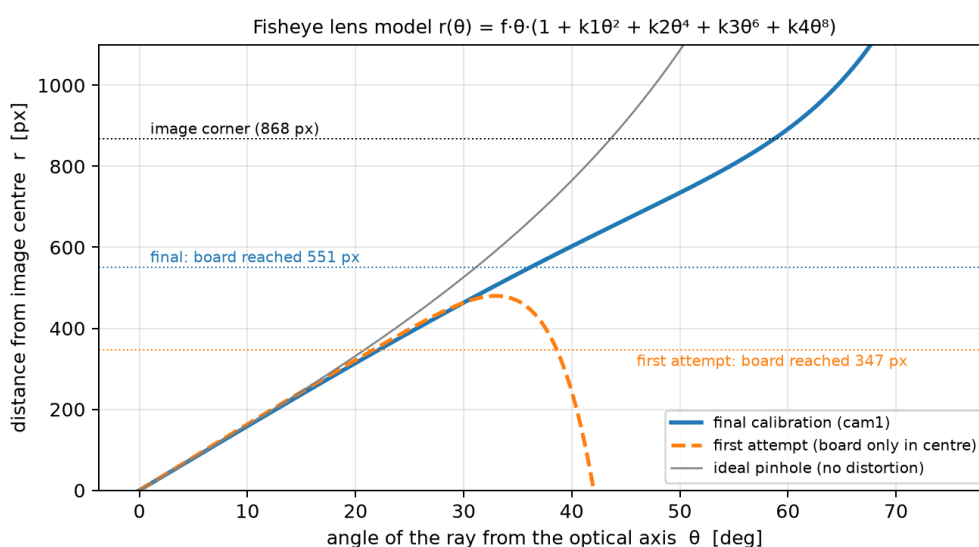


Figure 3 - Lens curves of cam1. Orange: first attempt (board only in the centre) - the polynomial folds back beyond the measured region. Blue: final calibration, monotonic over the whole image (measured up to 551 px, extrapolated beyond).

An automatic check was added: it reports how far from the centre the board was seen and whether the distortion curve stays monotonic, and prints **USABLE / NOT USABLE**. The board was then re-recorded closer to the cameras and moved over the image.

3.6 Final calibration results

camera	board frames found	RMS	f_x / f_y [px]	c_x, c_y [px]	board reach	work area covered
cam1	719	0.84 px	911 / 917	596, 490	63 %	99 %
cam3	595	0.95 px	932 / 939	620, 541	66 %	98 %
cam4	783	0.76 px	902 / 907	636, 505	64 %	96 %

The image corners still were not reached - but they show ceiling and walls. What matters is the area where the person moves. The framework measures it from the mopping video with background subtraction and checks how much of it lies inside the board-covered area: **96-99 %**. The fisheye and pinhole fits also agree now (principal point within 5-10 px, focal length within 1 %), and straight edges in the room become straight after undistortion (Figure 5).



Figure 4 - cam1: heat map of where people move in the mopping video; white outline = area covered by board points. 99 % of the motion is inside.



Figure 5 - cam4: original fisheye frame and undistorted versions (balance 0 / 0.5 / 1 = how much of the wide field of view is kept). Walls, cabinet and window frames become straight.

The calibrations, detected board points and report images are stored in the repository (calibration/session1/, git tag calib-session1); parameter values are listed in the appendix.

4. Step 2 - 2D pose estimation

4.1 Choice of models

The videos are processed offline, so **accuracy has priority over speed**. The state of the art was reviewed in October 2026:

model	keypoints	reported accuracy	role here
Sapiens2-0.8B (Meta, ICLR 2026)	308: body, feet, hands, face	Sapiens2-1B 80.4 mAP, 5B 82.3 mAP on a 308-keypoint in-the-wild benchmark (0.8B: not reported separately)	main model ; vision transformer at 1024 x 768 input
RTMW (rtmlib, ONNX)	133: COCO-WholeBody	70.1 AP (COCO-WholeBody)	fast baseline for cross-checking; also used by Pose2Sim
YOLO26x (Ultralytics)	person boxes	-	person detection + tracking
YOLO26x-pose	17 body	71.6 AP (COCO body)	not used: no foot points
SAM 3D Body (Meta, CVPR 2026)	3D mesh per image	single-view 3D	candidate for the later joint-angle stage

For the robot we keep a canonical set of **23 joints**: the 17 COCO body points (nose, eyes, ears, shoulders, elbows, wrists, hips, knees, ankles) plus big toe, small toe and heel on each foot. All other keypoints of the models (e.g. hands) are stored as well, for later use.

4.2 Finding the person who mops

YOLO26x detects every person in each raw frame and the BoT-SORT tracker links the detections over time into tracks. The office contains several seated people, so the mopper is selected as the track whose **feet travel the farthest** (bottom-centre of the box, median-smoothed). Trackers sometimes split one person into several tracks (occlusion); pieces are joined when one ends close in time (< 3 s) and space (< 200 px) to where the next begins. A preview image lets the user confirm the choice or force a track id.



Figure 6 - cam1, first 20 s: six tracks; the mopper (green, track 2) is chosen automatically, the five seated people (grey) are ignored.

4.3 Removing the fisheye distortion around the person - the virtual camera

Pose networks are trained on ordinary photographs. Near the edge of a fisheye image a person is bent and squeezed, which degrades the keypoints. Undistorting the whole frame is not possible for a wide fisheye (the borders would be stretched enormously). Instead, for every frame a **virtual pinhole camera** is placed at the same optical centre and rotated to look straight at the person:

- the person's bounding-box outline is converted to viewing rays with the calibration
- the virtual camera's axis points at the angular centre of these rays, with the image 'up' kept upright
- its focal length is chosen so the person fills the crop with a margin (the models add 25 % themselves)
- for every crop pixel the corresponding ray is projected through the real fisheye model, and the raw image is resampled there (a remapping table)
- keypoints found in the crop are converted back: crop pixel $\rightarrow$ ray $\rightarrow$ raw fisheye pixel. This round trip is exact to < 0.05 px (unit test).

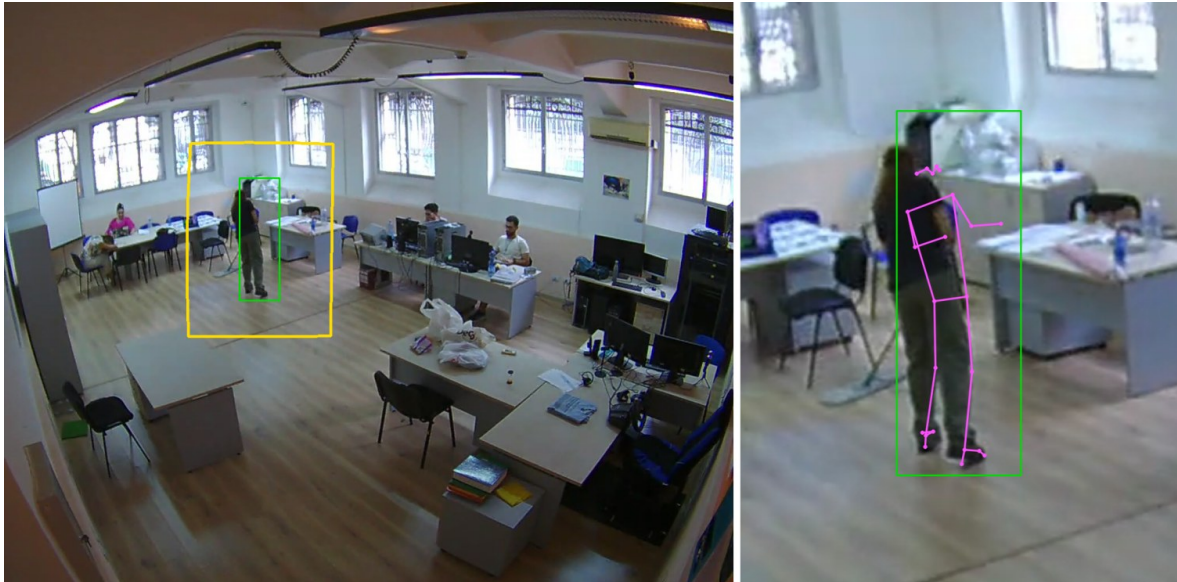


Figure 7 - Left: raw fisheye frame; green = detected person box, yellow = region seen by the virtual camera. Right: the rectified, upright 768 x 1024 crop given to the network, with the Sapiens2 skeleton.

4.4 Results on the real footage (cam1, first 20 s, 301 frames)

	Sapiens2-0.8B	RTMW
mean keypoint confidence	0.87	0.81
frame-to-frame jitter (median 2nd difference)	0.64 px	0.84 px
speed on RTX 2070 (8 GB), incl. flip test	0.83 frames/s	~30 frames/s
GPU memory	2.8 GB (fp16)	small

Without ground truth, the two independent models serve as a cross-check: they agree within **1-4 px on all 23 joints** (Figure 9). Sapiens2 is more confident and steadier, so it is the main model; RTMW remains as a fast fallback and sanity check. The decisive accuracy test comes after triangulation: the reprojection error of the 3D joints in all three cameras and the constancy of bone lengths over time.

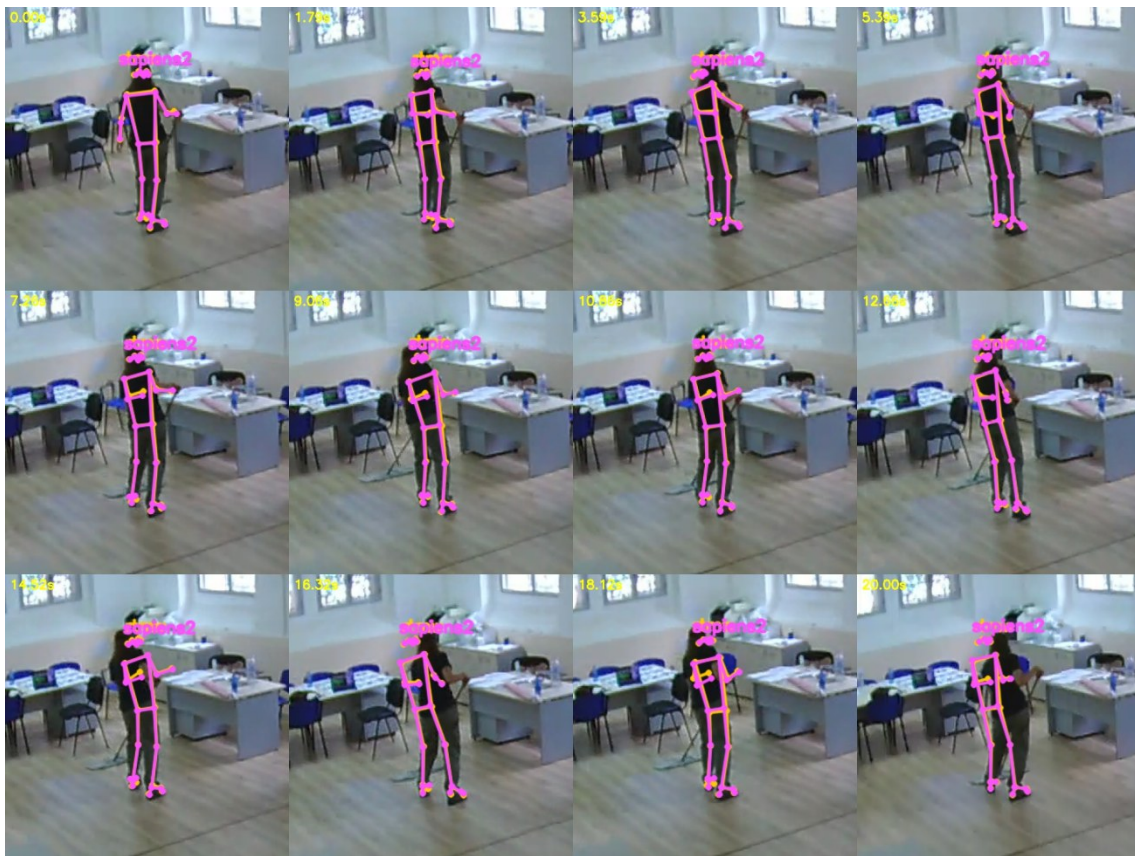


Figure 8 - cam1: twelve frames over 20 s, zoomed on the moppper. Pink = Sapiens2, orange = RTMW (mostly hidden under pink because they agree).

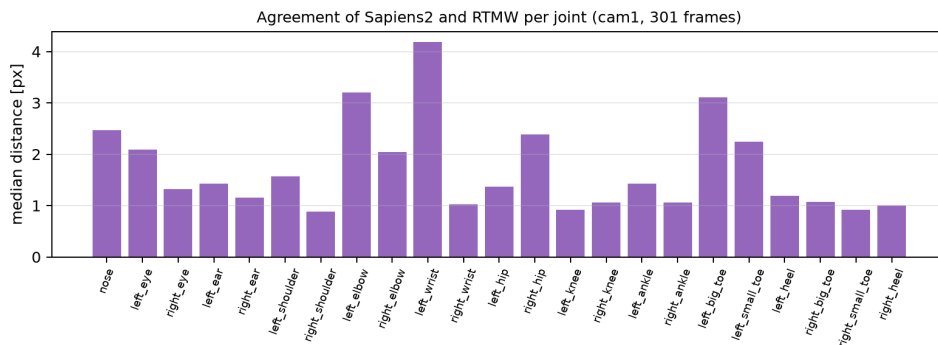


Figure 9 - Median distance between Sapiens2 and RTMW per joint (raw image pixels).

Problems found during testing and how they were solved

- The RTMW-x 'cocktail13' release placed the **left toes on the head** on this footage (hundreds of pixels from the ankle, with high confidence). Switched to the RTMW 'cocktail14' release; its feet agree with Sapiens2.
- RTMW returns unbounded confidence scores ($\sim 3-8$); they are mapped to 0-1 with $s/(1+s)$, raw values kept.
- A CPU-only ONNX Runtime package silently replaced the GPU version (same module name); it is now excluded and the GPU build is pinned to CUDA 12 to match PyTorch.

4.5 Computation

Each camera has $\sim 17\,800$ frames. On the laptop Sapiens2 would need about 6 hours per camera, so the full run is done on a stronger Windows GPU computer. The code is moved as one git bundle; two commands install everything and download the models; results are written in resumable chunks, so a run can be stopped and continued; only small result files (a few MB) come back. Each result file contains, per frame, the real timestamp, the 23 joints in raw fisheye pixels, their confidences, and all native keypoints of the model.

5. Step 3 - Triangulation (implemented, waiting for camera poses)

When the cameras' positions are known (extrinsics: rotation R and translation t of each camera, $X_{\text{cam}} = R \cdot X_{\text{world}} + t$), a joint seen in several cameras is reconstructed by the **Direct Linear Transform (DLT)**. Each detected keypoint is first undistorted to normalised coordinates (x, y) with the lens calibration. With $P = [R \mid t]$ the projection gives two linear equations per camera in the unknown homogeneous point X :

$$w \cdot (x \cdot P_3 - P_1) \cdot X = 0, w \cdot (y \cdot P_3 - P_2) \cdot X = 0$$

where P_i is the i -th row of P and w the detection confidence (weight). Stacking all cameras gives $A \cdot X = 0$, solved by singular value decomposition. Robustness: if a camera's reprojection error for a joint exceeds a threshold (e.g. a mis-detection), every leave-one-camera-out solution is computed and the most consistent one is kept; joints seen by fewer than two cameras are left empty.

Tests on a synthetic 3-camera fisheye rig: noise-free points are recovered to < 1 mm; with 0.5 px noise and 80 px errors injected into one camera for 10 points, the bad camera is rejected (one geometrically ambiguous case, where the error lies along an epipolar line, cannot be detected with only three cameras - temporal filtering will handle such cases).

6. Software and reproducibility

- Python package moppose, Python 3.12, managed with uv; one command-line tool with sub-commands (probe, board-check, calib-intrinsics, people, pose2d, pose-preview, pose-all, setup)
- 16 automated tests with synthetic data: a ChArUco board rendered through a known fisheye camera by ray casting (calibration must recover it), the 0.5 px correction, triangulation with missing/bad views, the virtual-camera round trip on the real cam1 calibration, and the mopper selection
- videos stay local; calibration results are versioned in git; every step writes visual reports so results can be checked by eye
- configuration in two small files: the board (board.yaml) and the session (which video belongs to which camera)

7. Next steps

step	what is needed	method
Extrinsic calibration	a reference object seen by all three cameras at the same time (e.g. markers or tape marks on the floor with measured positions)	mark the same points in each camera; solve each camera's pose (PnP) in one world frame (metres, Z up); check by triangulating the points back
Time synchronisation	a shared event (e.g. clap, light switch) or the motion itself	estimate the offset between recordings, refine by cross-correlating the 2D joint trajectories
3D landmarks	results of the two steps above	triangulate the 23 joints per frame, reject outliers, fill short gaps, smooth (low-pass filter)
Joint angles	3D landmarks	fit a skeleton with constant bone lengths; output angles for the robot

Known limitations. Calibration is extrapolated in the image corners (ceiling/walls, outside the work area). With three cameras, a joint occluded in two views cannot be triangulated in that frame. Left/right confusion of limbs is possible when the person is seen from behind; multi-view consistency will be used to detect it.

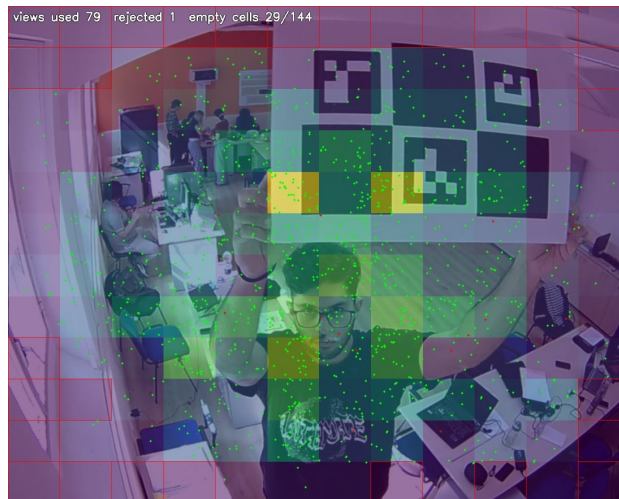
Appendix A - Calibration parameters (1280 x 1024, OpenCV fisheye model)

camera	f_x, f_y [px]	c_x, c_y [px]	k_1, k_2, k_3, k_4	RMS
cam1	911.43, 916.69	595.69, 490.38	-0.0967, -0.0368, -0.0092, 0.0669	0.84 px
cam3	931.55, 939.22	619.87, 540.50	-0.1114, 0.0846, -0.1493, 0.1057	0.95 px
cam4	901.87, 907.44	635.95, 505.31	-0.1276, 0.1752, -0.5372, 0.5481	0.76 px

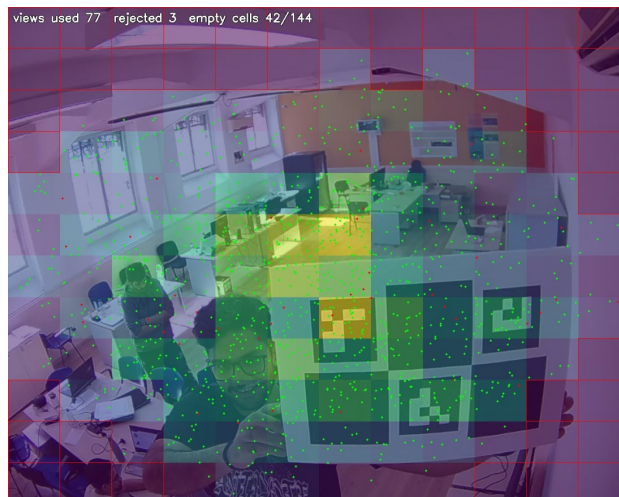
Appendix B - Main commands

```
uv sync --all-extras # install (PyTorch CUDA, models' dependencies)
uv run moppose setup # download Sapiens2, RTMW, YOLO26; GPU check
uv run moppose probe <videos> # real frame rate, dropped frames, contact sheet
uv run moppose board-check <video> # does board.yaml match the board?
uv run moppose calib-intrinsics --cam cam1 # lens calibration + report
uv run moppose people --cam cam1 # tracking, mopper selection
uv run moppose pose2d --cam cam1 # 2D pose (Sapiens2 + RTMW)
uv run moppose pose-all # everything for all cameras
```

Appendix C - Calibration coverage of the other cameras



cam3: board points (green) and coverage heat map; red cells = no board points.



cam4: same report.